Guía de Actividades Práctico-Experimentales Nro. 001

1. Datos Generales

Asignatura	Estructura de Datos
Ciclo	3ro A / B
Unidad	1: Estructuras de datos estáticas y dinámicas
Resultado de aprendizaje de la unidad	Modela el uso de memoria y diseña Tipos de Datos Abstractos (TDA) para comprender las diferencias de complejidad espacial, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	01
Título de la Práctica	Micro-TDAs, Abstracción de Datos y Análisis de Memoria en Motores V8
Nombre del Docente	Cristian Ramiro Narváez Guillén
Fecha	jueves 01 y viernes 02 de octubre 2026
Horario	Jueves 01 de octubre 10:30 – 11:30 Viernes 02 de octubre 11h30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

- **Modelar** Tipos de Datos Abstractos (TDA) simples utilizando clases y constructores en JavaScript moderno.
- **Medir** empíricamente el impacto en la memoria Heap (complejidad espacial S(n) al procesar estructuras de datos basadas en tipos primitivos frente a arreglos de objetos masivos.
- **Evaluar** mediante el análisis del recolector de basura (Garbage Collector) las decisiones de diseño físico y su impacto en la sostenibilidad computacional (Green Computing).

3. Materiales y Reactivos

- Software Node.js (versión LTS 20.x o superior) instalado en el sistema.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).

- Conexión a Internet estable para consultas a la documentación técnica oficial (ABI).
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).

4. Procedimiento / Metodología

Enfoque PID-2025:

- **ABP (Aprendizaje Basado en Problemas):** El equipo es contratado por un proyecto de geolocalización satelital sostenible. Deben cargar $N = 10^6$ coordenadas en memoria. El problema radica en que instanciar 10^6 objetos colapsa los límites del entorno. Se debe diseñar y medir la alternativa más eficiente.
- **ABI (Aprendizaje Basado en Investigación):** Los estudiantes deberán investigar cómo el motor de JavaScript (V8) reserva memoria contigua vs. memoria dinámica.
- **Modalidad:** Individual y discusión grupal posterior.

Tarea 1: Preparación del Entorno de Medición (Profiler)

Para medir la huella de memoria en JavaScript, utilizaremos el módulo integrado process.

1. Cree un nuevo directorio para el proyecto y abra su editor de texto.
2. Cree un archivo denominado profiler.js.
3. Implemente la función de medición de memoria RAM (Heap):

```
/**
 * Tarea 1: Utilidad para medir la memoria Heap en Megabytes (MB).
 * El Heap es el espacio de memoria dinámico donde V8 almacena objetos y variables.
 */
function getMemoryUsage() {
    const memoryData = process.memoryUsage();
    // Convertimos de bytes a Megabytes (MB) usando notación matemática estándar
    const heapUsedMB = Math.round(memoryData.heapUsed / 1024 / 1024 * 100) / 100;
    return heapUsedMB;
}

module.exports = { getMemoryUsage };
```

Tarea 2: Enfoque 1 - Modelado basado en Arreglo de Objetos (TDA)

En esta tarea, crearemos N objetos. Esta es la representación de más alto nivel de abstracción.

1. Cree un archivo llamado lab01_objetos.js.
2. Incluya el siguiente código, el cual creará $1,000,000$ de instancias de coordenadas.

```
const { getMemoryUsage } = require('./profiler');

const N = 1000000; // 1 millón de registros
const memoriaInicial = getMemoryUsage();

// TDA: Coordenada basada en Objetos
class CoordenadaObj {
    constructor(lat, lng) {
        this.lat = lat; // Número de punto flotante de 64 bits
        this.lng = lng;
    }
}

// Almacenamos en memoria
let coordenadas = [];
for (let i = 0; i < N; i++) {
    coordenadas.push(new CoordenadaObj(i * 0.1, i * -0.1));
}

const memoriaFinal = getMemoryUsage();
console.log(`[Enfoque Objetos] Memoria inicial: ${memoriaInicial} MB`);
console.log(`[Enfoque Objetos] Memoria final: ${memoriaFinal} MB`);
console.log(`[Enfoque Objetos] Consumo Neto: ${memoriaFinal - memoriaInicial} MB`);
```

3. Ejecute en su terminal: node lab01_objetos.js y anote el resultado neto.

Tarea 3: Enfoque 2 - Modelado basado en Arreglos de Primitivos paralelos (Data-Oriented Design)

1. Cree un archivo llamado lab01_primitivos.js.
2. Implemente la estructura de datos desacoplando la abstracción del objeto a arreglos paralelos unidimensionales fuertemente tipados (Float64Array).

```
const { getMemoryUsage } = require('./profiler');

const N = 1000000;
```

```
const memoriaInicial = getMemoryUsage();

// TDA Desacoplado: Uso de Typed Arrays para obligar a V8 a usar memoria contigua
// La capacidad total requerida es N elementos de 8 bytes (64 bits) cada uno
let lat = new Float64Array(N);
let lng = new Float64Array(N);

for (let i = 0; i < N; i++) {
        lat[i] = i * 0.1;
        lng[i] = i * -0.1;
}

const memoriaFinal = getMemoryUsage();
console.log(`[Enfoque Primitivos] Memoria inicial: ${memoriaInicial} MB`);
console.log(`[Enfoque Primitivos] Memoria final: ${memoriaFinal} MB`);
console.log(`[Enfoque Primitivos] Consumo Neto: ${memoriaFinal - memoriaInicial} MB`);
```

3. Ejecute en su terminal: node lab01_primitivos.js y anote el resultado.

Tarea 4: Investigación y Cálculo de Complejidad

Basado en los resultados, el estudiante debe aplicar ABI para investigar el porqué de la diferencia matemática.

1. Busque en la documentación oficial de V8 cómo se empaquetan las propiedades en los *Hidden Classes* de JavaScript.
2. Calcule teóricamente en LaTeX el espacio asintótico. Sabiendo que un Float64 pesa 8 bytes, demuestre por qué.

5. Resultados esperados:

- El estudiante obtendrá evidencia empírica en consola demostrando que el manejo de Tipos de Datos Abstractos complejos mediante instanciación masiva de objetos genera un *overhead* significativo en la memoria (*heap*), incrementando la latencia del recolector de basura. Se espera una diferencia de consumo sustancial a favor de los Typed Arrays (`Float64Array`).

6. Preguntas de Control:

- Responda las siguientes preguntas reflexionando sobre las causas fundamentales de los fenómenos computacionales observados:

1. **Causa Material:** ¿De qué está constituido fundamentalmente un objeto en la memoria del entorno de ejecución (V8), y en qué difiere la

"materia" de un objeto de la de un valor primitivo continuo en un `Float64Array`?

2. **Causa Formal:** ¿En qué medida la "forma" o estructura lógica que imponemos a un Tipo de Dato Abstracto (TDA) define y restringe los límites de su eficiencia física en el hardware?

3. **Causa Eficiente:** Al observar el consumo de memoria, ¿es el intérprete de JavaScript el verdadero artífice (causa eficiente) del desperdicio de recursos, o reside la responsabilidad primaria en la decisión arquitectónica del desarrollador?

4. **Causa Final:** ¿Cuál es el propósito último de abstraer los datos en la programación orientada a objetos si, en su esencia más fundamental, toda optimización crítica requiere retornar a la disposición contigua de bits y bytes en la memoria?

5. **Esencia y Existencia:** Si dos estructuras de datos (como el Enfoque 1 y el Enfoque 2) resuelven exactamente el mismo problema lógico arrojando los mismos resultados, pero existen físicamente ocupando volúmenes de energía y memoria drásticamente distintos, ¿son en esencia la misma solución? Argumente.

- ## Evaluación

Criterio Evaluado	**Puntaje Máximo**	**Evidencia Esperada**
1. Implementación TDA y Entorno	2 puntos	Capturas de pantalla con la ejecución correcta de los archivos .js en consola sin errores de sintaxis.
2. Medición y Tabulación de Datos	2 puntos	Tabla comparativa de consumos en MB entregada en el informe que demuestre las diferencias entre objetos y primitivos.
3. Análisis Matemático / Teórico	3 puntos	Explicación fundamentada del

(ABI)		*overhead* utilizando notación Big-O e investigación sobre *Hidden Classes* en V8.
4. Respuestas Aristotélicas	3 puntos	Respuestas a las preguntas de control que demuestren profundidad de análisis sobre la naturaleza lógica y física de los datos.
Total	10 puntos	

7. Bibliografía

1. P. W. Bible and L. Moser, *An Open Guide to Data Structures and Algorithms*. PALNI Open Press, 2023. [Online]. Available: https://open.palni.org/
2. Mozilla Developer Network, "Memory management", *MDN Web Docs*, 2023. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Memory_Management
3. O. Levin, *Discrete Mathematics: An Open Introduction*, Autoeditado, 2024.

8. Elaboración y Aprobación

- Elaborado por: nombre del Docente responsable de la práctica.
- Aprobado por: nombre del Director de Carrera.

Elaborado por	Cristian R. Narváez Guillén **Docente**	
Aprobado por	Edison L Coronel Romero **Director de Carrera**	